

Deciphering the DSA submission bottleneck

Diagnostic target
missing cycles under full-WQ pressure

Signal	Scale	Meaning
Unloaded submit	20 cycles	posted/admitted write; credits available
Single request	sub-μs visible completion	N00P 379 ns; 64B 575 ns
Full-loop async	8–10 Mops/s	closed-loop cost, not isolated submit

cheap submit ≠ cheap closed loop

Sources: submit-only study 2026-05-24; Understanding the Host Network; direct latency measurement.

Host path view: what to measure

Where credits live

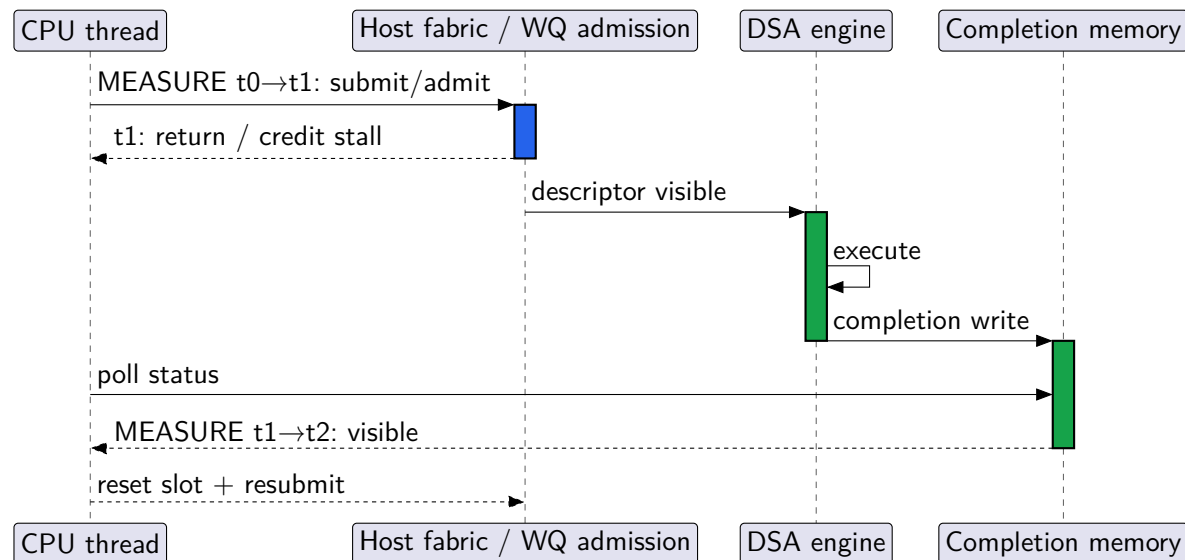
domain	processor / memory / peripheral path
credits	outstanding request budget
unloaded latency	cheap path when credits exist
wait	upstream work gets slowed

DSA implication

movdir64b returns	the WQ accepted the write
not proved	DSA execution
not proved	completion writeback
not proved	CPU can see completion
full-loop cost	first place that runs out of credits

Design rule: vary one pressure source → find first bend

The DSA loop: where timestamps attach



t0 → t1 submit / waiting

t1 → t2 work done + visible

lifeline fill = measured region

■ t0→t1 submit interval

■ t1→t2 done-and-visible interval

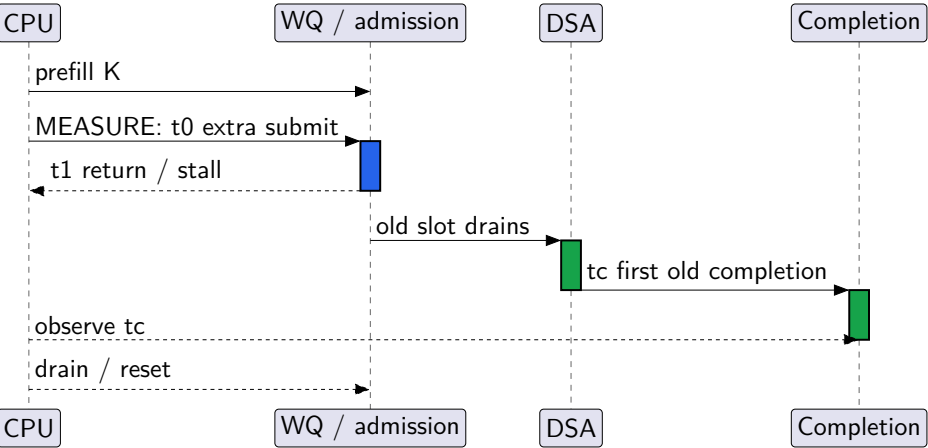
DSA traffic classes

DSA action	Host path	What it tests
movdir64b submit	CPU → peripheral posted write	t0→t1 submit slowdown
DSA source read	Peripheral → memory read	payload-read pressure
DSA destination write	Peripheral → memory write	write-buffer / fabric pressure
DSA completion write	Peripheral → memory write, cacheline-scale	completion becomes visible
CPU poll/reset	CPU memory read/write	cacheline ownership; slot reuse

Exp. 3: add classes **Exp. 5:** collide classes

Experiment 1: submit gets slow when WQ is full?

Setup



Readout

submit latency vs. occupancy	
lifeline fill = measured region	
■ extra-submit timer	
■ first old-slot completion wait	
flat	submit credits remain
knee near 128	WQ/device credits
early knee	store / posting / IIO / portal
full-WQ tail	wait for device progress

x	K = 0,32,64,96,112,120,124,127,128
y	extra-submit cycles
hold fixed	op, NUMA, WQ, core

Fill WQ more → see when submit gets slow.

Why run it

why	fill the WQ, then time one extra submit
learn	when the extra submit gets slow
record	K, submit cycles, first old done time

If submit stays fast until the queue is nearly full, the 20-cycle number is still real. If it gets slow early, the host/WQ path is already running out of credits.

Experiment 1: why run it

Simple Rust sketch

rust

```
for &k in &[0, 32, 64, 96, 112, 120, 124, 127, 128] {  
    bench.reset_slots();  
    bench.prefill(k, Op::Noop);  
  
    let t0 = ticks();  
    bench.submit_one(Op::Noop);  
    let t1 = ticks();  
  
    let first_done = bench.poll_first_old();  
    bench.drain_all();  
  
    record("credit-knee", k, t1 - t0, first_done);  
}
```

These are simple calls; the real runner still sets descriptors, fences, and checks completions.

Experiment 1 result: admission bends before WQ size

first plateau

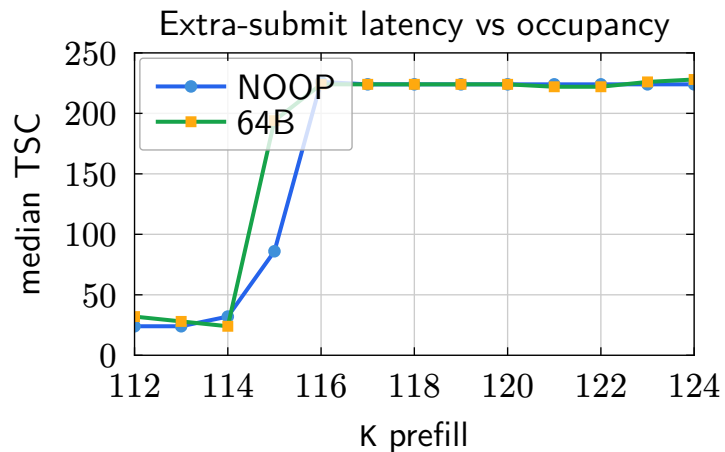
$K \approx 116$

extra submit

224–226 ticks

nominal WQ

128



Source: docs/report/benchmarking/019.submit_occupancy_one_extra_2026-05-26.md

Attribution

one-submit vs K bends early

Cheap through $K \leq 114$: 24 - - 32 TSC ticks.

Transition at $K=115$: NOOP 86, 64B 194 ticks.

Plateau begins at $K=116$; 64B follows NOOP, so payload is not the first bend.

Experiment 2: trace the wall region

Setup

Question

Experiment 1 found the submit wall near requests 115–117. Are early requests already complete before that wall?

CPU submit order; tracing starts at poll_offset



lifeline fill = measured region

☐ completion records tracked: comp[1..4]

☐ first traced submit offset

☐ admission-wall region from Experiment 1

Measured at every traced index

submit_tsc[i]	latency of the i th MMIO submit
poll_tsc[i,j]	cost to read completion j after submit i
visible prefix	how many of comp[1..4] are visible contiguously
visible count	how many of comp[1..4] are visible in any order

poll step = 1; offsets = 96, 112, 115

The first poll at an offset is the boundary check; later polls intentionally trace the active-observation path.

Poll offset: where do we start looking?

We keep the marker at request 1 and vary only the first submit index where polling begins.

offset 96

Starts well before the wall. Useful for seeing how active polling changes the subsequent submit stream.

offset 112

Boundary check before the wall is expected to bite. If comp[1] is visible here, DSA completed early work before the wall.

offset 115

Checks the wall itself. The next submit can expose the wall while the first poll shows what had already completed.

decision axis: first visible completion index vs. submit-cost wall index

Experiment 2: why run it

Simple Rust sketch

rust

Why run it

concern	cheap submits may be only upstream buffering
learn	whether comp[1..4] are already visible by index 112/115
cost	how expensive the observation poll is

If comp[1] is visible at the first poll at index 112, then request 1 was not waiting for the wall to become device-visible.

```
for i in 1..=n {  
    let t0 = rdtscp();  
    submit(desc[i]);  
    let t1 = rdtscp();  
  
    if i >= poll_offset {  
        trace.submit[i].push(t1 - t0);  
  
        for j in 1..=4 {  
            let p0 = rdtscp();  
            let status = comp[j].status();  
            let p1 = rdtscp();  
            trace.poll[i][j].push(status, p1 - p0);  
        }  
    }  
}
```

Poll cost is part of the measurement

Poll states

NONE poll

CPU reads a completion line before hardware has written success.

first SUCCESS poll

First observed device-written success on that line.

hot SUCCESS poll

Later reads after the completion line is already hot to the CPU.

Why keep submit latency too?

Polling can stretch the loop, but submit latency still tells us when the wall appears in the traced run.

first poll	boundary observation at the configured offset
later polls	active-observation trace
submit trace	does the wall still appear?

record a list, not only one aggregate

Experiment 2 result: completions are visible by the wall

offset 112

comp[1..4]

visible at first poll

offset 115

comp[1..4]

visible at wall

correctness

160 / 160

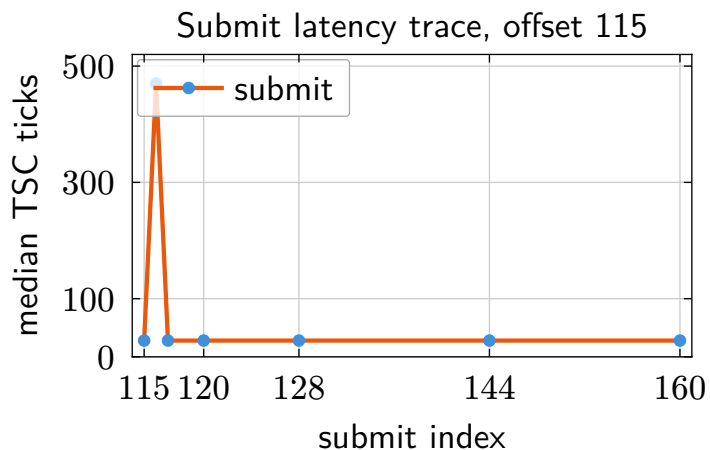
completed; 0 missing/errors

First-poll summary

offset	prefix	comp1	poll
96	0	0.016	78 ticks
112	4	1.000	72 ticks
115	4	1.000	70 ticks

At offsets 112 and 115, the first four completions are already visible at the first observation.

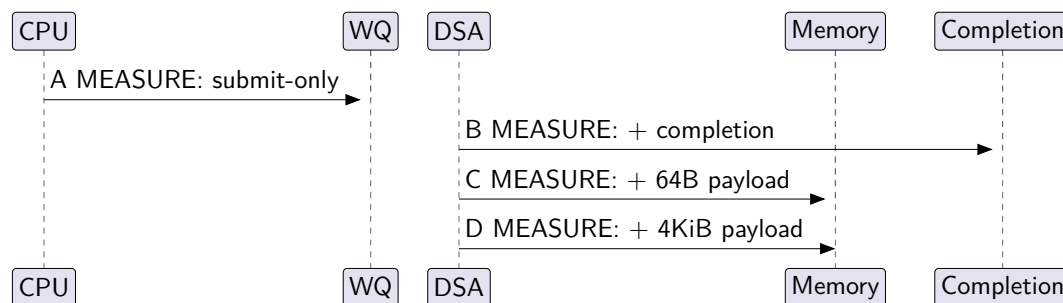
Hot success polls settle near 22 TSC ticks after the first success observation.



Source: docs/report/benchmarking/021.submit_marker_trace_2026-05-27.md.

Experiment 3: add one traffic type at a time

Traffic ladder



A	submit-only / no completion
B	completion-bearing NOOP
C	64B memmove
D	4KiB memmove

Readout

same curves; one class added

lifeline fill = measured region

■ submit-only path

■ completion path

■ 64B payload path

■ 4KiB payload path

A bends	submit alone
B adds	completion write / seen by CPU
C/D add	payload DMA domain
D unique	fabric / bandwidth / locality

Experiment 3: why run it

Simple Rust sketch

rust

Why run it

why	change one thing per run
learn	which added work makes it slow
compare	same WQ, core, window, NUMA node

This is the simple ladder: submit only, then completion writes, then payload reads/writes. The first new slowdown says what to check next.

```
let classes = [
    Op::SubmitOnly,
    Op::Noop,
    Op::Memcpy { bytes: 64 },
    Op::Memcpy { bytes: 4096 },
];

for op in classes {
    for window in [1, 8, 32, 64, 96, 128] {
        bench.reset_slots();

        let curve = bench.run_window(op, window, reps);
        record_curve("traffic", op, window, curve);
    }
}
```

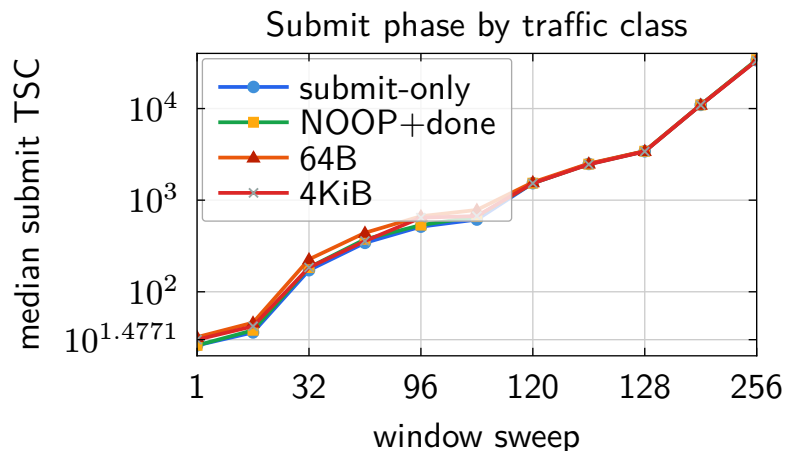
The runner changes only the operation class; all pacing and measurement code stays shared.

Experiment 3 result: payload does not move the knee

first jump
window 120

large by
window 128

64B vs NOOP
same knee



Source: docs/report/benchmarking/020.submission_bottleneck_experiments_2026-05-27.md, Experiment 3.

Throughput readout

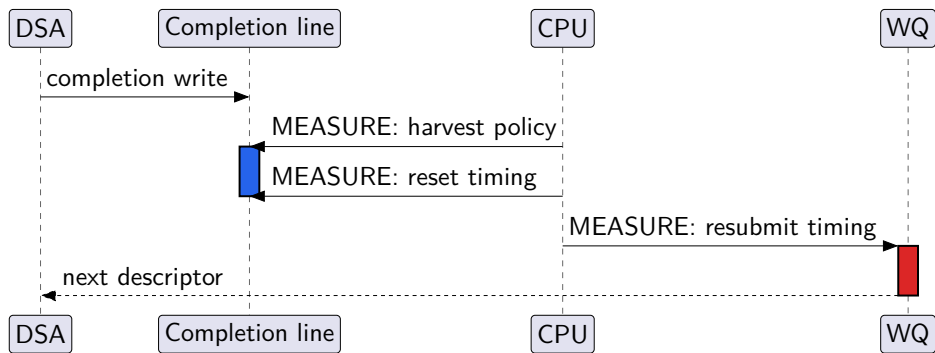
NOOP+completion holds around 9.1 Mops/s at high windows.

64B memmove is close to NOOP at high window: 9.08 Mops/s at 256.

4KiB lowers throughput to 6.44 Mops/s, but the submit knee stays put.

Experiment 4: completion handling loop

Reuse loop



layout	packed / padded
poll	scan-all / round-robin
reset	none / delayed / immediate
submit	per-completion / batch harvest

Readout

throughput sensitivity to reuse policy

lifeline fill = measured region
completion-line harvest
completion-line reset
WQ resubmit

padding helps	ownership / false sharing
scan hurts	poll load stream
batch helps	burstiness / cache churn
no movement	lower-level path

Experiment 4: why run it

Simple Rust sketch

rust

Why run it

why	CPU completion handling can be the slow part
learn	which CPU policy changes throughput
record	ops/sec, polls/done, reset-to-submit cycles

If padding, scan order, or batch harvesting changes the curve, fix the CPU completion path before changing hardware submission.

```
for policy in [
    PackedScan,
    PaddedRoundRobin,
    PollOnly,
    DelayedReset,
    BatchHarvest,
] {
    bench.configure(policy);
    bench.fill_window(128, Op::Noop);

    while bench.elapsed() < run_for {
        let done = bench.harvest(policy);
        bench.reset_done(done, policy);
        bench.resubmit(done, Op::Noop);
    }

    record("reuse", policy, bench.ops_per_sec(), bench.polls_per_done());
}
```

Experiment 4 result: reuse policy is not the first bend

NOOP best

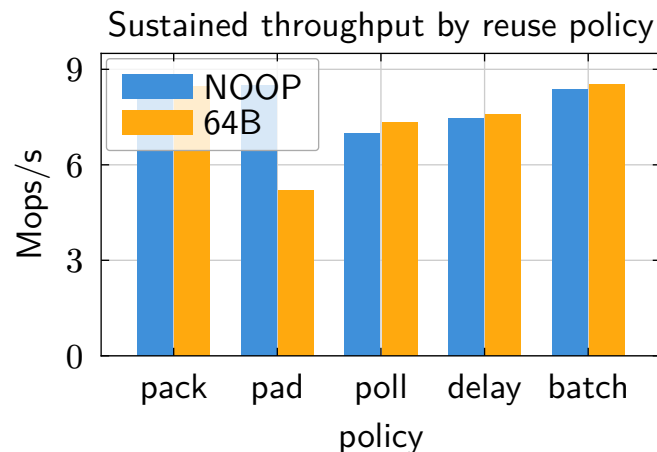
8.51 Mops/s

memmove64 best

8.53 Mops/s

correctness

0 missing



Attribution

policy moves throughput, not the admission knee

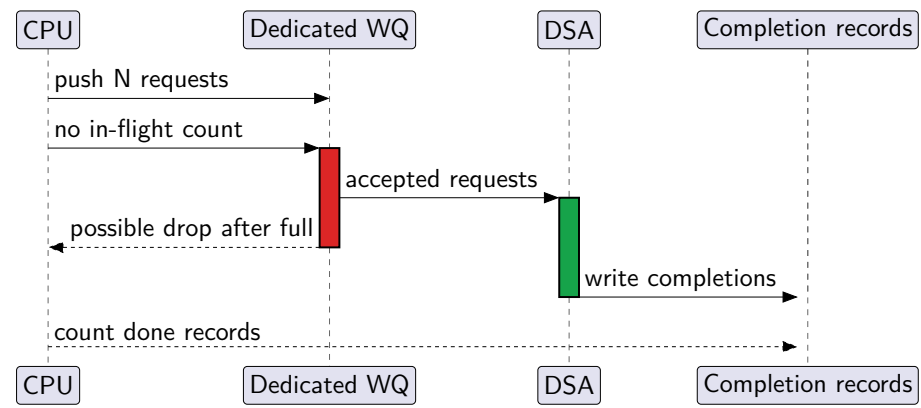
Packed scan, padded round-robin, and batch harvest stay in the same rough band for NOOP.

Every policy completed 1,000,000 operations with missing = 0 and errors = 0.

Source: docs/report/benchmarking/020.submission_bottleneck_experiments_2026-05-27.md, Experiment 4.

Experiment 5: push without counting

Setup



Readout

completed count vs. pushed count

lifeline fill = measured region
■ push past WQ depth
■ accepted requests complete

same count	no loss seen in this run
missing ids	some pushes were dropped
loss after depth	WQ full behavior
SWERROR set	hardware noticed bad submit

N	depth/2, depth, 2×depth, 4×depth
op	N00P first; optional 64B memmove
rule	do not poll while pushing

This intentionally stops counting in-flight work to test whether requests can disappear.

Why run it

why	movdir64b does not tell us “WQ accepted it”
learn	whether pushing without counting loses requests
record	pushed, completed, missing ids, SWERROR

If pushed count is larger than completed count, then we need in-flight tracking before trusting any throughput number from a dedicated WQ.

Experiment 5: why run it

Simple Rust sketch

rust

```
for n in [depth / 2, depth, 2 * depth, 4 * depth] {  
    bench.reset_unique_slots(n);  
  
    for id in 0..n {  
        bench.submit_no_wait(id, Op::Noop);  
    }  
  
    let done = bench.wait_done_until(timeout);  
    let missing = bench.missing_ids(0..n);  
    let swerr = bench.read_software_error();  
  
    record("no-count-push", n, done.len(), missing, swerr);  
}
```

Use one completion slot per pushed request; otherwise slot reuse can hide loss.

Experiment 5 result: blind push did not lose descriptors

max burst

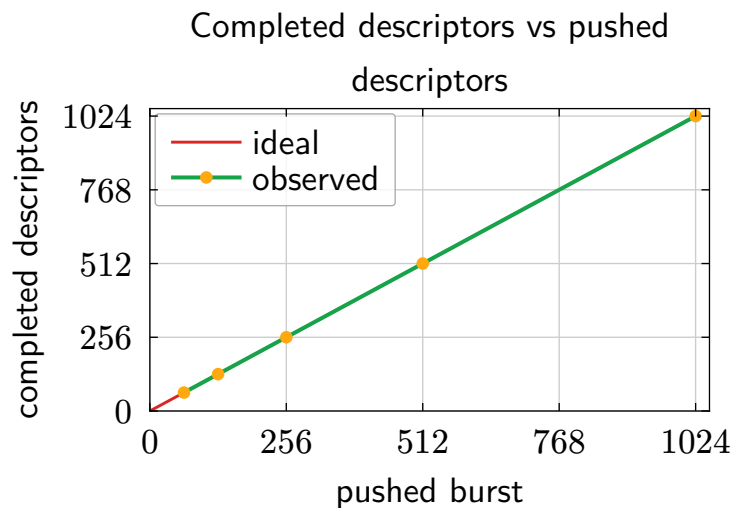
1024

completed

all

missing/errors

0 / 0



Source: docs/report/benchmarking/020.submission_bottleneck_experiments_2026-05-27.md, admission probe / Experiment 5.

What it rules out

request disappearance is not the measured failure mode

Over-depth submission still completes every distinct descriptor in this run.

Submit TSC still grows: 418 at burst 64, 217986 at burst 1024.

Attribution rule

First bend	Credit domain	Next probe
one-submit vs K	submit / WQ accept	knee depth vs WQ size
trace marker	early completions visible by wall	separate progress from admission
late marker	posting / completion delay	IIO / WQ credits
reuse variants	poll / reset / cacheline	reuse redesign
no-count push loses	missing completions	restore in-flight tracking

Find first bend → name the credit domain.

Measured answer: WQ admission is the first bottleneck

practical bound	nominal WQ	not explained by
< 116 outstanding	128 entries	64B / reuse / loss

Probe	Observed result	Implication
one-submit vs K	K≈116 plateau at 224- -226 ticks	admission / credits bend first
marker trace	comp[1..4] visible at first poll for offsets 112 and 115	DSA progressed before/at the wall
traffic ladder	same submit knee for submit-only, NOOP, 64B, 4KiB	payload does not move first bend
reuse policy	best policies stay around 8.5 Mops/s	completion loop not the first cause
blind push	1024/1024 complete; missing/errors 0/0	not descriptor loss

Interpretation: keep batch-size-1/no-batch logical concurrency below the observed admission threshold, not the nominal WQ size.

Sources: docs/report/benchmarking/019.submit_occupancy_one_extra_2026-05-26.md; docs/report/benchmarking/020.submission_bottleneck_experiments_2026-05-27.md; docs/report/benchmarking/021.submit_marker_trace_2026-05-27.md.